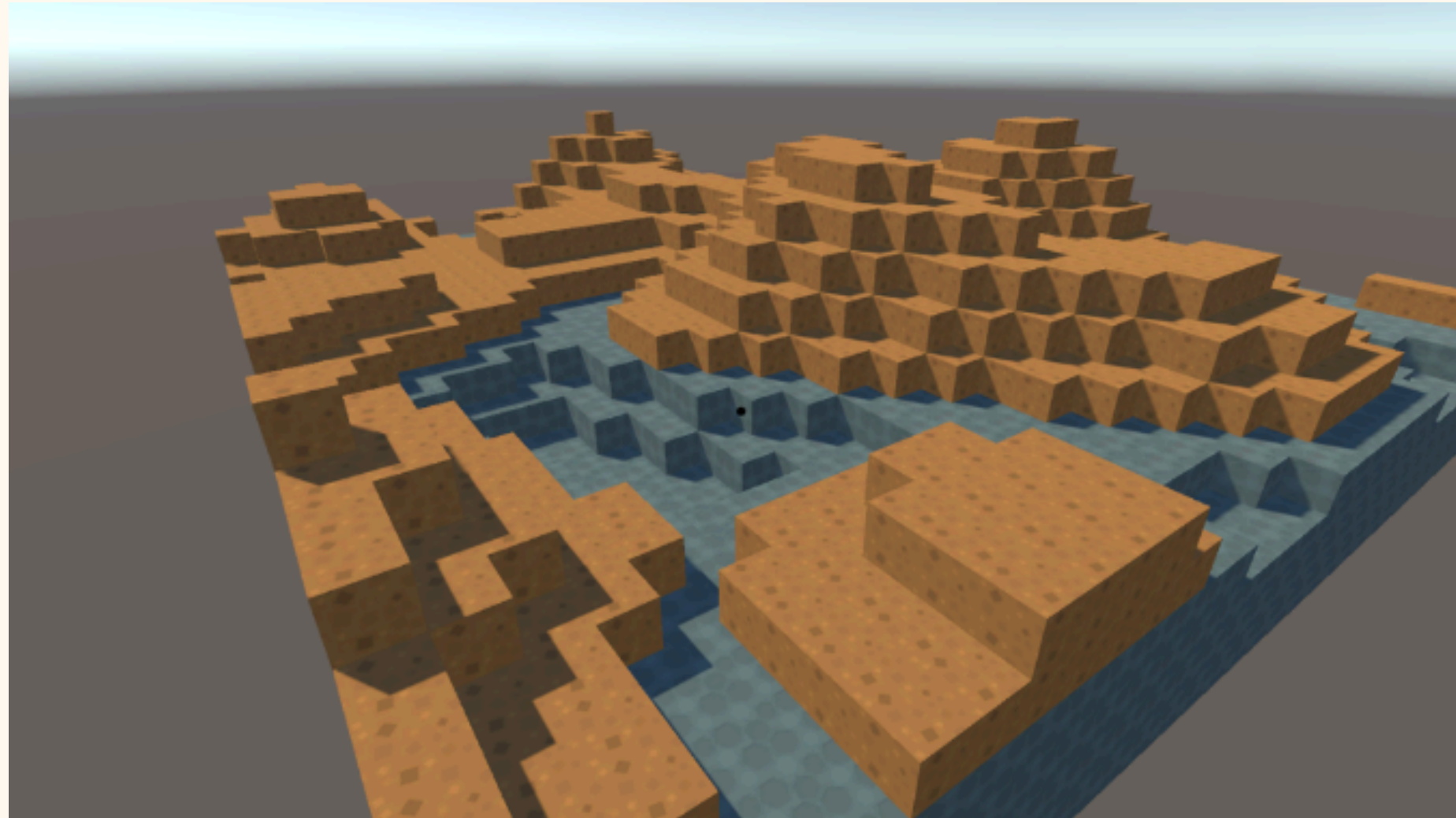


LogiCube

Optimisations

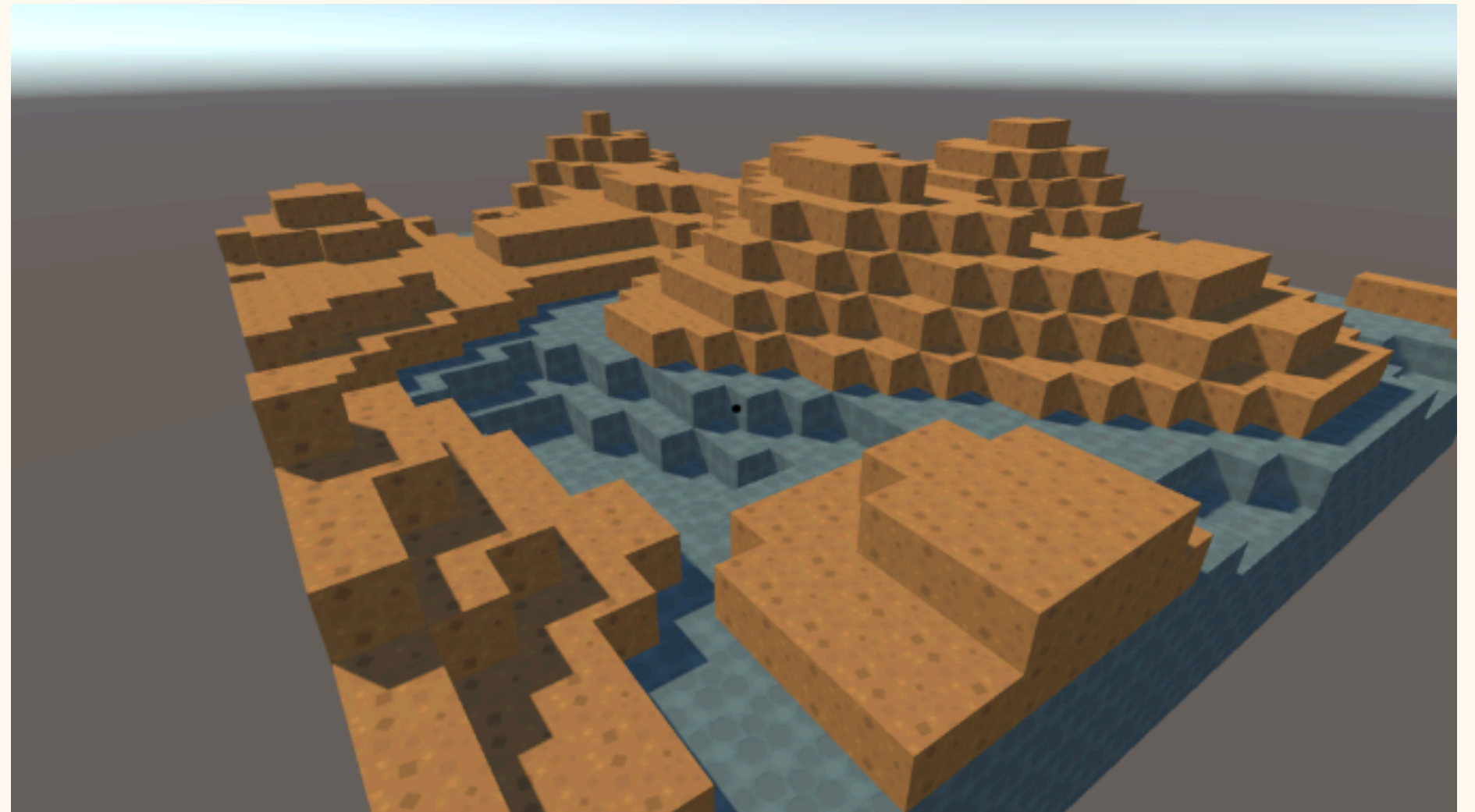
Nous avons un terrain !



Mais

**Chaque cube est un
GameObject.**

**Dans un monde de 32 par 32
blocs avec une moyenne de de
5 de haut, cela nous fait 5120
cubes !**



**Imaginez un monde de 500 par 500, ce qui n'est rien
comparé à Minecraft .-.**

Deux petits tricks

**Dans un premier temps, pourquoi ne pas
juste montrer les blocs visibles ?**

**On réduit de presque 5 le nombre de
cubes !**

Deux petits tricks

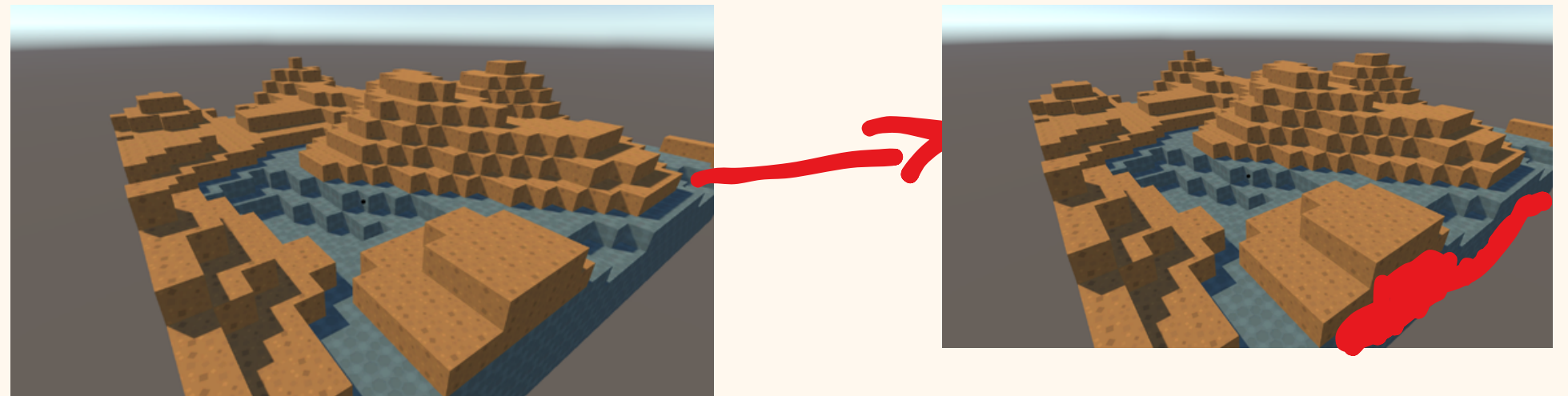
Cela nous fait 1024 cubes !

**Ce qui fait encore pas mal de
GameObject a gérer..**

La deuxième astuce est de n'afficher qu'un objet qui représente la surface.

On appelle ça un Mesh.

On va venir scanner la surface de notre monde et répliquer l'élévation avec une surface unique.



Créez un nouveau Script BlockType

```
18 references
public enum BlockType
{
    1 reference
    Air = 0, // vide
    2 references
    Dirt = 1, // terre
    3 references
    Stone = 2, // pierre
}

0 references
public static class BlockTypeExtensions
{
    2 references
    public static bool IsSolid(this BlockType type) => type != BlockType.Air;
}
```

Un Enum

```
18 references
public enum BlockType
{
    1 reference
    Air = 0, // vide
    2 references
    Dirt = 1, // terre
    3 references
    Stone = 2, // pierre
}

0 references
public static class BlockTypeExtensions
{
    2 references
    public static bool IsSolid(this BlockType type) => type != BlockType.Air;
}
```

Un énum est une sorte de liste d'objet. On l'utilise ici pour définir la valeur de nos blocs

Un Enum

```
18 references
public enum BlockType
{
    1 reference
    Air = 0, // vide
    2 references
    Dirt = 1, // terre
    3 references
    Stone = 2, // pierre
}

0 references
public static class BlockTypeExtensions
{
    2 references
    public static bool IsSolid(this BlockType type) => type != BlockType.Air;
}
```

Un énum est une sorte de liste d'objet. On l'utilise ici pour associer la valeur Air au nombre 0, ect

Is_Solid ?

```
0 references
public static class BlockTypeExtensions
{
    2 references
    public static bool IsSolid(this BlockType type) => type != BlockType.Air;
}
```

**IsSolid est une fonction qui vas nous permettre de savoir si un
bloque est solide (Pierre, Terre) ou non (Air)**

**C'est une fonction bool, donc elle vas nous renvoyer True ou False
si le bloc est différent d'un bloc d'Air.**

Créez un script FaceCulling

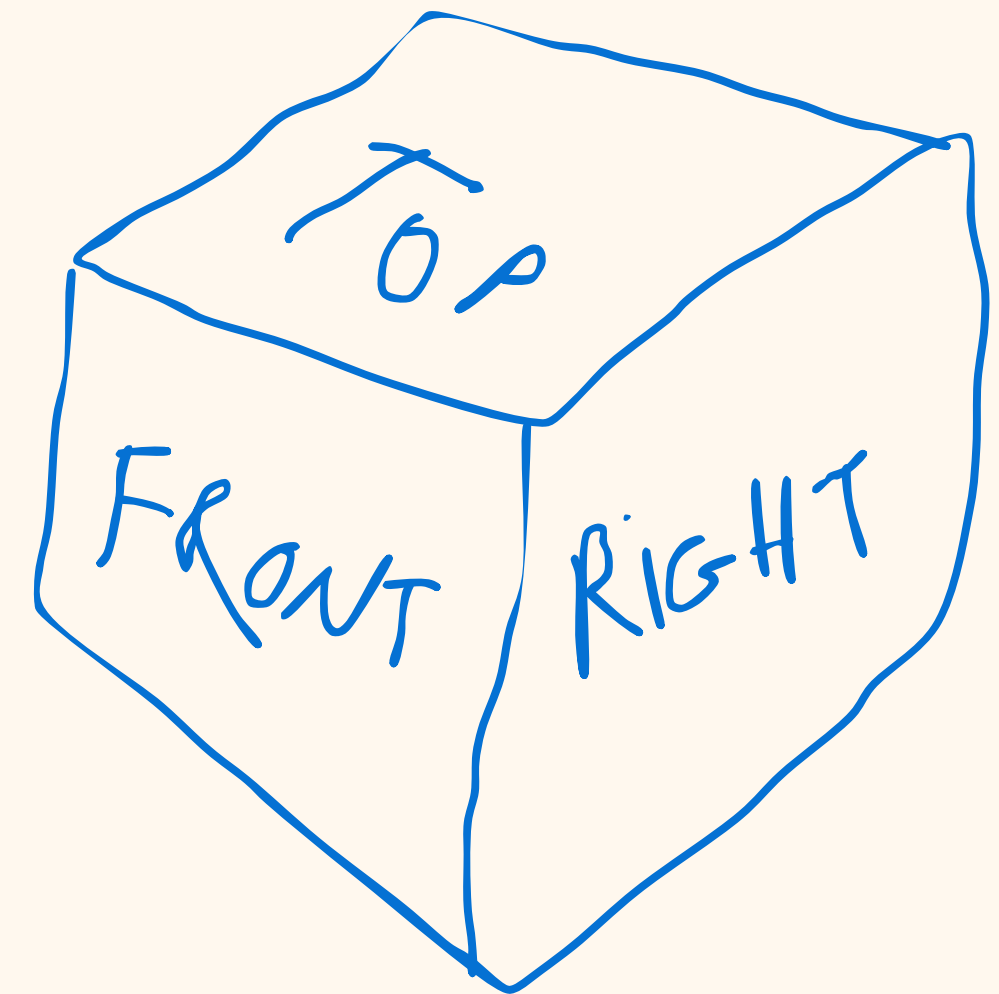
Le face culling est la technique qui nous permet de savoir quelles face du bloc on doit afficher, entre autre celles qui sont en contact avec de l'air, et donc visible !

Notre fonction IsSolid sera très utile !

```
public static readonly Vector3Int[] DirectionVectors =  
{  
    Vector3Int.up,           // Top  
    Vector3Int.down,         // Bottom  
    Vector3Int.right,        // Right  
    Vector3Int.left,         // Left  
    new Vector3Int(0, 0, 1),  // Front  
    new Vector3Int(0, 0, -1), // Back  
};
```

On crée une liste avec les 6 directions autour de notre bloc

Ça vas nous être utile pour connaître la position du bloc collé à la face que l'ont veux.



**Ex : Mon bloc est en $x=2, y=2, z=1$
Le bloc collé à la face front est donc
en : $x=2 + 0, y = 2 + 0, z= 1 + (-1)$**

Cette fonction vas nous donner une liste de toutes les faces à afficher pour un bloc

```
public static List<FaceDirection> GetVisibleFaces(Vector3Int pos, Dictionary<Vector3Int, BlockType> blocks)
{
    var visible = new List<FaceDirection>();

    for (int i = 0; i < 6; i++)
    {
        Vector3Int neighborPos = pos + DirectionVectors[i];

        bool neighborIsSolid = blocks.TryGetValue(neighborPos, out var neighbor)
                                && neighbor.IsSolid();

        // Si le voisin n'est pas solide → la face est exposée → on la garde
        if (!neighborIsSolid)
            visible.Add((FaceDirection)i);
    }

    return visible;
}
```

- On créer une liste pour toutes les faces visibles
- On boucle ensuite sur toutes les faces d'un bloc (donc 6)
- Et on regarde si le bloc à côté de la face i est solide
- Si il ne l'est pas, on ajoute la face dans la liste visible

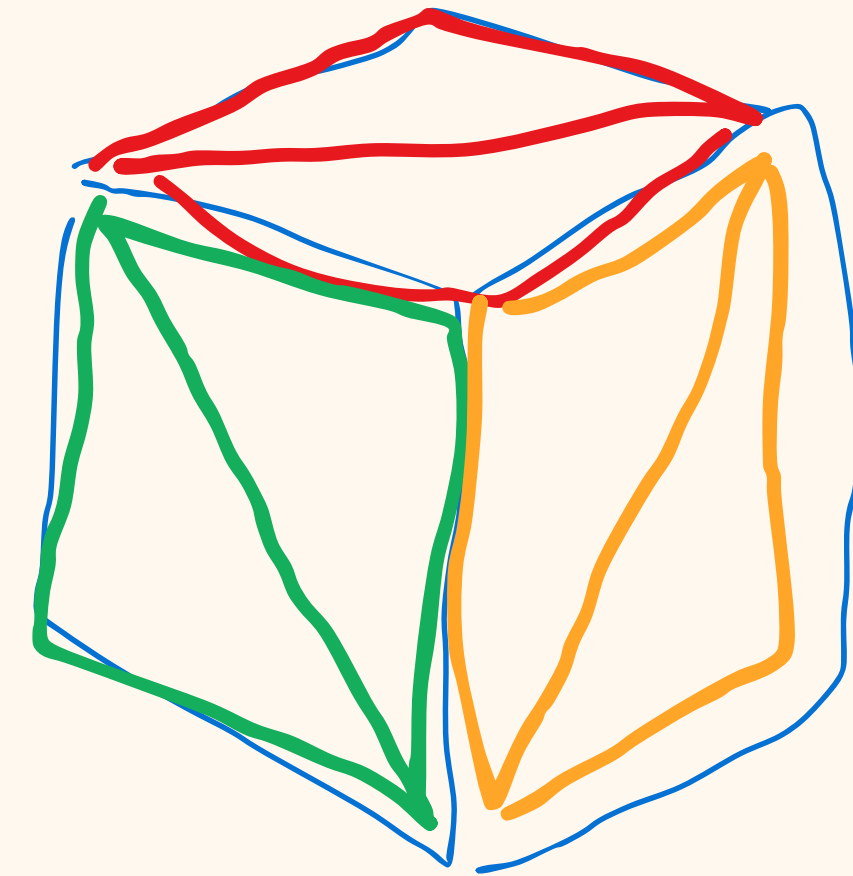
Créez un script ChunkMeshBuilder

```
MESH BUILDER

Théorie :
Un Mesh Unity = 3 tableaux :
    vertices → les points 3D dans l'espace
    triangles → des groupes de 3 indices qui forment
                des triangles
    uvs      → les coordonnées de texture

Pour chaque face visible (reçue du FaceCuller), on ajoute
4 points et 2 triangles au mesh.

    3 — 2
    | \ | triangle 1 : 0-1-2
    0 — 1 triangle 2 : 0-2-3
```



**Pour ce bloc ou on ne voit que
3 faces, on aura 6 triangles**

Le script est un peut complexe, mais en deux mots :

- 1. On définit les 4 coins de chaque face d'un cube (haut, bas, droite, gauche, devant, derrière).**
- 2. On définit les coordonnées UV pour placer la texture sur une face.**
- 3. On parcourt chaque face visibles de chaque bloc.**
- 4. Pour chaque face, on appelle AddFace pour ajouter la face à notre Mesh**
- 5. Quand on a toutes les faces, on créé un objet Mesh**
- 6. Pour chaque triangle du mesh, on récupère son matériau**
- 7. Et on calcule la forme de notre mesh avce les bons matériaux**

Ce qui nous donne l'impression d'un monde de bloc !

Les Dictionnaires

Type de donnée qui lie une **clé** à une **valeur**

```
mon_dico = <"pomme": 4, "bannane":8>
```

Ici, on a un dictionnaire avec comme clé le nom d'un fruit, et sa quantité comme valeur.

Pour récupérer le nombre de pommes :

```
mon_dico.get("ma_clé")
```


WorldGenerator

```
Assets > Scripts > WorldGenerator.cs > WorldGenerator > Generate(int) : Dictionary<Vector3Int, BlockType>
4 public class WorldGenerator : MonoBehaviour
5 {
6     1 reference | Unity Serialized Field
    public int chunkSize = 32;
7     2 references | Unity Serialized Field
    public float noiseScale = 0.1f;
8     1 reference | Unity Serialized Field
    public int terrainHeight = 10;
9
10    2 references
    public Dictionary<Vector3Int, BlockType> Generate(int seed)
11    {
12        var prng = new System.Random(seed);
13        var offset = new Vector2(prng.Next(-100000, 100000),
14                                prng.Next(-100000, 100000));
15
16        var blocks = new Dictionary<Vector3Int, BlockType>();
17        int half = chunkSize / 2;
18
19        for (int x = -half; x < half; x++)
20            for (int z = -half; z < half; z++)
21            {
22                float nx = (x + offset.x) * noiseScale;
23                float nz = (z + offset.y) * noiseScale;
24                int height = Mathf.FloorToInt(Mathf.PerlinNoise(nx, nz) * terrainHeight);
25
26                for (int y = 0; y <= height; y++)
27                {
28                    var type = (y > 3) ? BlockType.Dirt : BlockType.Stone;
29                    blocks[new Vector3Int(x, y, z)] = type;
30                }
31            }
32
33        return blocks;
34    }
35 }
```

On crée un dictionnaire qui stock pour chaque position le type de bloc

On a comme clé unique la position x,y, et z du bloc via le Vecteur3.

Et comme valeur le type

Plus de Instantiate, on ajoute juste au dict avec mon_dico[ma_clé] = ma_valeur;

WorldManagerScript

Variables d'instances

Avant

```
public class WorldManagerScript : MonoBehaviour
{
    public GameObject[] allPrefabs;
    public GameObject player;
    public WorldGenerator generator;
    private string savePath;

    private readonly Dictionary<Vector3Int, GameObject> worldBlocks = new();
    private readonly Dictionary<Vector3Int, string> placed = new();
    private readonly Dictionary<Vector3Int, string> removed = new();

    private WorldSaveData loaded;

    void Awake()
    {
        savePath = Path.Combine(Application.persistentDataPath, "world.json");
        loaded = LoadSaveFileOrCreate();
    }
}
```

Maintenant

```
1 reference | ⓘ Unity Script (1 asset reference)
public class WorldManagerScript : MonoBehaviour
{
    [Header("Références")]
    2 references | ⓘ Unity Serialized Field
    public WorldGenerator generator;
    6 references | ⓘ Unity Serialized Field
    public GameObject player;

    [Header("Matériaux")]
    2 references | ⓘ Unity Serialized Field
    public Material dirtMaterial;
    1 reference | ⓘ Unity Serialized Field
    public Material stoneMaterial;

    11 references
    private Dictionary<Vector3Int, BlockType> blocks = new();
    4 references
    private int seed;

    8 references
    private GameObject chunkG0;

    3 references
    private string SavePath => Path.Combine(Application.persistentDataPath, "save.json");
}
```

WorldManagerScript

Fonction start

Avant

```
void Start()
{
    // 1) Regénère le monde de base
    generator.Generate(loaded.seed, this);

    // 2) Applique le delta
    ApplyDelta(loaded);

    // 3) Player
    if (loaded.player != null && player != null)
    {
        player.transform.position = loaded.player.position;
        player.transform.rotation = loaded.player.rotation;
    }
}
```

Maintenant

```
0 references | Unity Message
void Start()
{
    var save = LoadOrCreate();
    seed = save.seed;

    // 1) On génère le monde de base avec la seed
    blocks = generator.Generate(seed);

    // 2) On remet les modifications du joueur (blocs posés/détruits)
    ApplyDelta(save);

    // 3) On construit le rendu 3D
    RebuildChunk();

    // 4) On replace le joueur où il était
    if (save.player != null && player != null)
    {
        player.transform.position = save.player.position;
        player.transform.rotation = save.player.rotation;
    }
}
```

WorldManagerScript

fonction applyDelta

Avant

```
private void ApplyDelta(WorldSaveData save)
{
    // 1) remove
    foreach (var b in save.removedBlocks)
    {
        if (worldBlocks.TryGetValue(b.position, out var go) && go != null)
        {
            if (GetBlockTypeFromGO(go) == b.blockType)
                RemoveBlockVisual(b.position);
        }
        else
        {
            RemoveBlockVisual(b.position);
        }
    }

    // 2) place
    foreach (var b in save.placedBlocks)
        PlayerPlacedBlock(b.position, b.blockType);
}
```

Maintenant

```
1 reference
private void ApplyDelta(WorldSaveData save)
{
    foreach (var b in save.removedBlocks)
        blocks.Remove(b.position);

    foreach (var b in save.placedBlocks)
        blocks[b.position] = System.Enum.Parse<BlockType>(b.blockType);
}
```

- Tous les blocs sont enregistrés dans un seul dico
- on parse l'enum pour obtenir le type de bloc

WorldManagerScript

- on boucle sur tous les blocs,
- on regarde quel face du bloc doit être visible
- on stock le tout dans un dictionnaire qui mémorise pour chaque blocs les faces visibles
- on consruit le mesh sur base des faces visibles

```
3 references
private void RebuildChunk()
{
    // Étape 1 : Face Culling – on trouve quelles faces sont visibles
    var visibleFaces = new Dictionary<Vector3Int, List<FaceDirection>>();
    foreach (var (pos, type) in blocks)
    {
        if (!type.IsSolid()) continue;
        var faces = FaceCuller.GetVisibleFaces(pos, blocks);
        if (faces.Count > 0)
            visibleFaces[pos] = faces;
    }

    // Étape 2 : Mesh Builder – on construit le mesh avec seulement ces faces
    var mesh = ChunkMeshBuilder.Build(visibleFaces, blocks, out var subMeshOrder);

    // Étape 3 : On associe un matériau à chaque type de bloc
    var materials = new Material[subMeshOrder.Length];
    for (int i = 0; i < subMeshOrder.Length; i++)
        materials[i] = GetMaterial(subMeshOrder[i]);

    // Étape 4 : On applique le tout à un seul GameObject
    if (chunkGO == null)
    {
        chunkGO = new GameObject("Chunk");
        chunkGO.AddComponent<MeshFilter>();
        chunkGO.AddComponent<MeshRenderer>();
        chunkGO.AddComponent<MeshCollider>();
    }
    chunkGO.GetComponent<MeshFilter>().mesh = mesh;
    chunkGO.GetComponent<MeshRenderer>().materials = materials;
    chunkGO.GetComponent<MeshCollider>().sharedMesh = mesh;
}
```

WorldManagerScript

poser/détruire un bloc

Avant

```
public void PlayerPlacedBlock(Vector3 pos, string blockType)
{
    Vector3Int p = Vector3Int.FloorToInt(pos);

    removed.Remove(p);
    placed[p] = blockType;

    RemoveBlockVisual(p);

    var prefab = FindPrefabByName(blockType);
    if (prefab != null)
    {
        var go = Instantiate(prefab, (Vector3)p, Quaternion.identity);
        worldBlocks[p] = go;
    }
}
```

Maintenant

```
1 reference
public void PlaceBlock(Vector3 worldPos, BlockType type)
{
    var p = Vector3Int.FloorToInt(worldPos);
    blocks[p] = type;
    RebuildChunk();
}

1 reference
public void DestroyBlock(Vector3 worldPos)
{
    var p = Vector3Int.FloorToInt(worldPos);
    if (!blocks.ContainsKey(p)) return;
    blocks.Remove(p);
    RebuildChunk();
}
```

- on retire simplement du dico
- on re fait le rendu du chunk

WorldManagerScript

Sauvegarder le monde

```
public void SaveWorld()
{
    var save = new WorldSaveData { seed = seed };
    var baseBlocks = generator.Generate(seed);

    // Ce que le joueur a ajouté
    foreach (var (pos, type) in blocks)
        if (!baseBlocks.ContainsKey(pos))
            save.placedBlocks.Add(new BlockData { position = pos, blockType = type.ToString() });

    // Ce que le joueur a détruit
    foreach (var (pos, type) in baseBlocks)
        if (!blocks.ContainsKey(pos))
            save.removedBlocks.Add(new BlockData { position = pos, blockType = type.ToString() });

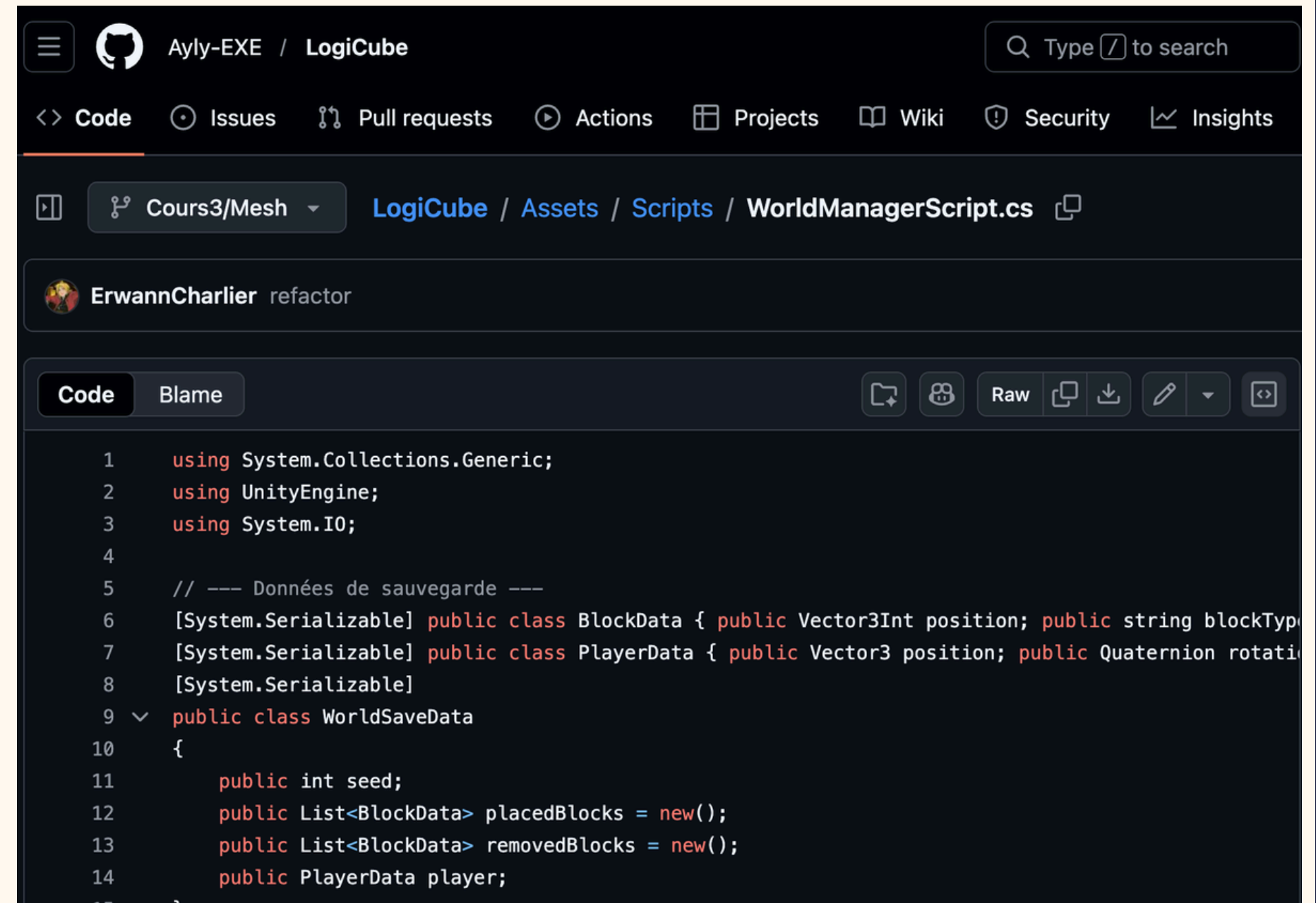
    if (player != null)
        save.player = new PlayerData
        {
            position = player.transform.position,
            rotation = player.transform.rotation
        };

    File.WriteAllText(SavePath, JsonUtility.ToJson(save, true));
}
```

WorldManagerScript

- On a passé en revue les fonctions les plus importantes de ce script.
- Il vous manque peut-être encore quelques fonctions utilitaires en fonction de votre implémentation.
- Vous pouvez les copier depuis GitHub, mais assurez-vous de bien les comprendre avant de les utiliser.
- ps: n'hésitez pas à poser des questions si quelque chose n'est pas clair :)

<https://github.com/Ayly-EXE/LogiCube/blob/Cours3/Mesh/Assets/Scripts/WorldManagerScript.cs>



The screenshot shows the GitHub interface for the repository 'Ayly-EXE / LogiCube'. The file path is 'Cours3/Mesh / LogiCube / Assets / Scripts / WorldManagerScript.cs'. A commit by 'ErwannCharlier' with the message 'refactor' is selected. The code editor shows the following C# code:

```
1  using System.Collections.Generic;
2  using UnityEngine;
3  using System.IO;
4
5  // --- Données de sauvegarde ---
6  [System.Serializable] public class BlockData { public Vector3Int position; public string blockType;
7  [System.Serializable] public class PlayerData { public Vector3 position; public Quaternion rotation;
8  [System.Serializable]
9  public class WorldSaveData
10 {
11     public int seed;
12     public List<BlockData> placedBlocks = new();
13     public List<BlockData> removedBlocks = new();
14     public PlayerData player;
15 }
```


PlayerAction

obtenir position d'un bloc (depuis ray cast) Maintenant

Avant

```
// Get the voxel position for block placement (only adjacent face)
public Vector3 GetBlockPlacementPosition(RaycastHit hit)
{
    // Get the hit block position (floor to voxel)
    Vector3 hitBlockPos = new Vector3(
        Mathf.FloorToInt(hit.transform.position.x),
        Mathf.FloorToInt(hit.transform.position.y),
        Mathf.FloorToInt(hit.transform.position.z)
    );

    // Determine which face was hit
    Vector3 normal = hit.normal;

    // Place the new block adjacent along the axis of the normal
    Vector3 placePos = hitBlockPos;

    if (Mathf.Abs(normal.x) > 0.5f) placePos.x += normal.x > 0 ? 1 : -1;
    else if (Mathf.Abs(normal.y) > 0.5f) placePos.y += normal.y > 0 ? 1 : -1;
    else if (Mathf.Abs(normal.z) > 0.5f) placePos.z += normal.z > 0 ? 1 : -1;

    return placePos;
}
```

```
1 reference
public Vector3 GetBlockPlacementPosition(RaycastHit hit)
{
    Vector3 insideBlock = hit.point - hit.normal * 0.5f;
    Vector3Int hitBlockPos = Vector3Int.FloorToInt(insideBlock);
    return hitBlockPos + Vector3Int.RoundToInt(hit.normal);
}
```

- Avant : hit.transform.position donnait directement la position du bloc touché.
- Maintenant : le raycast touche le chunk, donc il faut recalculer quel bloc est visé.
- hit.point donne le point exact où le rayon touche la surface du mesh.
- hit.normal donne la direction de la face touchée (haut/bas/droite,..)
- hit.point - hit.normal * 0.5f décale le point vers l'intérieur du bloc touché pour s'assurer que le bloc appartient bien à cette face.
- FloorToInt(...) convertit cette position en coordonnées entières du bloc touché.
- + hit.normal donne la position du bloc voisin où placer le nouveau bloc.

PlayerAction

poser/détruire bloc

Avant

```
void Update()
{
    if (Input.GetMouseButtonDown(0))
    {
        RaycastHit? hit = GetHit();
        if (hit.HasValue)
        {
            Vector3 blockPos = GetBlockPlacementPosition(hit.Value);
            //Instantiate(blockPrefab, blockPos, Quaternion.identity);
            worldManager.PlayerPlacedBlock(blockPos, blockPrefab.name);
            Debug.Log("Block placed at: " + blockPos);
        }
        else
        {
            Debug.Log("Nothing hit");
        }
    }

    if (Input.GetMouseButtonDown(1))
    {
        RaycastHit? hit = GetHit();
        if (hit.HasValue)
        {
            GameObject hitObject = hit.Value.collider.gameObject;

            if (hitObject.CompareTag("Block"))
            {
                //Destroy(hitObject);
                worldManager.PlayerDestroyedBlock(hitObject);
                Debug.Log("Block destroyed: " + hitObject.name);
            }
            else
            {
                //
            }
        }
    }
}
```

Maintenant

```
0 references | ☑ Unity Message
void Update()
{
    // Clic gauche = poser un bloc
    if (Input.GetMouseButtonDown(0))
    {
        RaycastHit? hit = GetHit();
        if (hit.HasValue)
        {
            Vector3 placePos = GetBlockPlacementPosition(hit.Value);
            worldManager.PlaceBlock(placePos, BlockType.Stone);
        }
    }

    // Clic droit = détruire un bloc
    if (Input.GetMouseButtonDown(1))
    {
        RaycastHit? hit = GetHit();
        if (hit.HasValue)
        {
            Vector3 insideBlock = hit.Value.point - hit.Value.normal * 0.5f;
            worldManager.DestroyBlock(insideBlock);
        }
    }
}
```